Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ” Informații invizibile: forensics și stenografie ”

A efectuat : st. gr.SI – 264

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Scopul acestei lucrări este dezvoltarea a două instrumente de analiză forensică în Python: un decoder universal capabil să identifice și să decodifice automat straturi (base64, hex, URL).Deasemenea, și să execute atacuri de tip brute-force/dicționar pentru chei XOR și hash-uri de parole, respectiv o unealtă de analiză a fișierelor.

1. DECODE

```python
# A1

import base64
import re
from urllib.parse import unquote

s = "Salut"
b = s.encode()  # bytes
print(b, b[0])  # b'Salut' 83

print(b.hex())
b_from_hex = bytes.fromhex(b.hex())
print("Bytes din hex:", b_from_hex)
```

Figura 1.1/A1

```python
# A2
def ghici_strat(s :{__contains__, fullmatch} ):
    if "%" in s:
        return "url"
    if re.fullmatch( pattern: r"[0-9a-fA-F]+", s) and len(s) % 2 == 0:
        return "hex"
    if re.fullmatch( pattern: r"[0-9a-zA-Z+/=]+", s):
        return "base64"
    return "necunoscut"


print(ghici_strat("%2Fetc"))  # url
print(ghici_strat("48656c6c6f"))  # hex
```

Figura 1.2/A2

```python
# A3

def desfa(s :{__contains__, fullmatch, encode} ):
    strat = ghici_strat(s)
    if strat == "base64":
        return strat, base64.b64decode(s)
    elif strat == "hex":
        return strat, bytes.fromhex(s)
    elif strat == "url":
        return strat, unquote(s).encode("utf-8")

    return strat, s.encode("utf-8")
```

Figura 1.3/A3

```python
# A4

def citibil(b):
    return len(b) > 0 and all(32 <= c < 127 for c in b)

val = "NTM1OTU3NTk="
for _ in range(8):
    strat, b = desfa(val)

    if citibil(b):
        print(f"Găsit text citibil [{strat}]:", b.decode())
        break
    else:
        try:
            val = b.decode("utf-8")
        except UnicodeDecodeError:
            break
```

Figura 1.4/ A4

```python
# A5

date = open("probe/xor.bin", "rb").read()
for k in range(256):
    clar = bytes(b ^ k for b in date)
    if citibil(clar):
        print("cheie: ", k, clar.decode())

# cheie:  128 Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}
```

Figura 1.5/A5

```python
# A6

import hashlib

tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256

for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
    # TODO: daca hashlib.md5(cuv.encode()).hexdigest()
    if hashlib.md5(cuv.encode()).hexdigest() == tinta:
        print("gasit:",cuv)
        break
```

Figura 1.6/A6

```python
# A7

import argparse, json
from pathlib import Path

p = argparse.ArgumentParser()
p.add_argument("intrare")
p.add_argument( *name_or_flags: "--fisier", action="store_true")
a = p.parse_args()

# TODO: daca a.fisier, citeste octetii din cale; altfel trateaza intrarea ca sir
if a.fisier:
    cale = Path(a.intrare)
    if not cale.is_file():
        print(f"Eroare: Fisierul '{a.intrare}' nu există.")
        exit(1)
    date = cale.read_bytes()
else:
    date = a.intrare.encode("utf-8")

strat = "text_sau_format_detectat"
try:
    rezultat = date.decode("utf-8", errors="ignore")
except Exception:
    rezultat = date.hex()

# TODO: salveaza {strat, rezultat} in arsenal/forensics/fisa.json
cale_iesire = Path("fisa.json")
cale_iesire.parent.mkdir(parents=True, exist_ok=True)

date_json = {
    "strat": strat,
    "rezultat": rezultat
}

with open(cale_iesire, "w", encoding="utf-8") as f:
    json.dump(date_json, f, indent=4, ensure_ascii=False)

# python decode.py probe/mesaj.txt --fisier ( comanda de rulare in terminal)
```

Figura 1.7/A7

2. FORENSICS

```python
# Extinderea B1-B4 / forensics.py


# B1

from pathlib import Path

def tip_real(cale):
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    if cap.startswith(b"%PDF"):
        return "pdf"
    #       b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    if cap.startswith(b"\x89PNG"):
        return "png"
    if cap.startswith(b"\xff\xd8"):
        return "jpeg"
    #       b"PK\x03\x04" -> "zip/docx"
    if cap.startswith(b"\x03\x04"):
        return "zip/docx"
    return cap.hex()
print(tip_real("probe/foto.jpg"))
```

Figura 2.1/ B1

```python
# B2

import re
date = open("probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
for m in re.findall( pattern: rb"[-~]{4,}",date):
    text = m.decode("latin1")
    print(text)
```

Figura 2.2/ B2

```python
# B3

from PIL import Image
from PIL.ExifTags import TAGS
img  = Image.open("probe/foto.jpg")
exif = img._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id),"=",val)
# TODO: ia blocul GPS (tag 34853) si tipareste

gps_info = exif.get(34853)
if gps_info:

    lat_data = gps_info.get(2)
    lat_ref = gps_info.get(1)

    long_data = gps_info.get(4)
    long_ref = gps_info.get(3)

    #       latitudinea si longitudinea
print(f"Latitudine: {lat_data},{lat_ref}")
print(f"Longitudine: {long_data},{long_ref}")
```

Figura 2.3/ B3

```python
# B4

date = open("probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")

# TODO: daca i != -1:
if i!= -1:
    #   open("gasit.zip","wb").write(date[i:])
    arhiva = open("probe/gasit.zip", "wb").write(date[i:])

#   print("arhiva extrasa de la octetul", i)
    print(f"Arhiva extrasa de la octetul : {arhiva}")
    print("Arhiva extrasa de la octetul: ",i)

# apoi dezarhiveaza si vezi ce era inauntru
    dezarhivarea = open("probe/gasit.zip", "rb").read()
    print(f"Continutul dezarhivarii:",dezarhivarea)

#x00secret.txtFLAG{ascuns_prin_carving}
```

Figura 2.4/ B4

3. STEGANOGRAFIE

```python
# B5

from PIL import Image

img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.get_flattened_data())
mesaj = b"FLAG{ascuns_in_pixeli}"

biti = "".join(f"{x:08b}" for x in mesaj)
canal = []
for p in px:
    canal.extend(list(p))

# TODO: pentru fiecare bit, ia un canal si pune
for i,bit in enumerate(biti):
    canal[i] = (canal[i] & ~1) | int(bit)

px_nou = []
for i in range(0,len(canal),3):
    px_nou.append((canal[i], canal[i+1],canal[i+2]))

# TODO: img.putdata(px_nou); img.save("stego.png")
img.putdata(px_nou)
img.save("stego.png")
```

Figura 3.1/ B5

```python
# B6

px = list(Image.open("stego.png").convert("RGB").get_flattened_data())
biti = ""

for (r, g, b) in px:
# TODO: aduna ultimii biti (r&1, g&1, b&1)
    biti += str(r & 1) + str(g & 1) + str(b & 1)
octeti = bytearray()

# TODO: grupeaza cate 8 biti -> octeti -> text
for i in range(0, len(biti), 8):
    octet = int(biti[i:i+8], 2)

#opreste-te la primul octet 0 (final de mesaj)
    if octet == 0:
        break
    octeti.append(octet)

mesaj_secret = octeti.decode("utf-8", errors="replace")
print(mesaj_secret)
```

Figura 3.2/ B6

CONCLUZIE

În urma realizării acestui laborator, am dezvoltat două unelte esențiale de analiză forensics și steganografie. Aceste module am construit un algoritm capabil sa desfacă straturi de decodificare si sa execute atacuri de tip BRUTE FORCE, pentru decriptarea mesajelor protejate de algoritmi precum XOR. De asemenea, am dobândit competente privind extragerea metadatelor de tip Exif si coordonatelor GPS.